ACADEMIC PROJECT REPORT

COURSE OUTCOMES: CO4, CO5, CO6 | BLOOM'S TAXONOMY: L2, L3

MediTrack: Smart Hospital Care System with MongoDB & AI Vector Search

A Modern Document-Oriented Healthcare Platform with Schema Validation, Atomic Array Telemetry, Complex Aggregations & Clinical Vector Search

Problem Statement:	Q1: Design and implement a MongoDB application for a real-world problem with validation, CRUD, PyMongo, and Aggregation.
Domain & Architecture:	Healthcare Informatics, Document NoSQL, Atlas Vector Database, Full-Stack Web
Primary Technologies:	MongoDB 6.0+, PyMongo 4.6+, Python (Flask), Atlas Vector Search, Chart.js
Target Evaluation:	Final Year B.Tech Computer Science & Engineering (30 Marks Evaluation)
Submission Date:	October 2026

Department of Computer Science & Engineering
Autonomous Engineering Institute | Accredited by NBA & NAAC

1. Abstract & Real-World Relevance

Modern healthcare facilities generate high-velocity, heterogeneous patient records spanning continuous vital telemetry, dynamic prescription schedules, and multi-departmental physician consultations. Traditional Relational Database Management Systems (RDBMS) force this dynamic data into rigid tabular structures, requiring 8 or more normalized tables for a single clinical encounter. This leads to severe JOIN bottlenecks during critical lookups and complex transaction locking under high concurrent write loads.

Real-World Clinical Relevance:

- **Zero-Join Medical Chart Retrieval:** Emergency room doctors retrieve a patient's complete diagnostic history, vitals trend, and medications in a single $O(1)$ disk read.
- **Atomic Telemetry Streaming:** Attending nurses append second-by-second vital readings using MongoDB's atomic `$push` operator without locking the main record.
- **Strict Clinical Data Integrity:** MongoDB's engine-level `$jsonSchema` blocks malformed entries (e.g. negative heart rates or invalid IDs) before storage.
- **AI Vector Database for Clinical RAG:** 64-dimensional dense semantic embeddings stored directly inside patient documents allow physicians to query symptoms in natural language and discover matching historical cases.

> ***Key Architectural Insight:*** *In healthcare, data that is accessed together should be stored together. MongoDB's hybrid document model co-locates high-frequency vitals and prescriptions with patient demographics, completely eliminating the relational N+1 query problem.*

2. Database & Document Schema Design

MediTrack avoids the classic antipattern of either 'embedding everything' or 'normalizing everything'. Instead, it implements an enterprise **Hybrid Data Model**:

Collection	Modeling Pattern	Cardinality & Engineering Justification
patients	Embedded Document (Vitals & Prescriptions)	**1-to-Few relationship.** Vitals and medications are bounded in size (<100 entries per hospital stay) and accessed together in 99% of requests. Embedding enables atomic $push updates with 0 joins.
doctors	Referenced Entity (`assigned_doctor_id`)	**1-to-Many unbounded relationship.** A physician treats thousands of patients over years. Referencing avoids duplicate doctor profiles and prevents update anomalies.
departments	Referenced Entity	Tracks shared hospital physical infrastructure (bed quotas, floor locations, and heads of departments).

BSON Document Structure (patients collection):

```
{
  "_id": ObjectId("6712ab..."),
  "patient_id": "PAT-1001",                    // Unique Index, Regex validated
  "name": "Robert Miller", "age": 62, "gender": "Male", "blood_group": "A+",
  "department": "Cardiology", "status": "Admitted",
  "assigned_doctor_id": "DOC-201",             // Foreign Reference -> doctors
  "vitals_history": [                          // Embedded Array (High Locality)
    { "timestamp": ISODate(...), "heart_rate": 105, "blood_pressure": "145/95", "oxygen_level"
: 94 }
  ],
  "prescriptions": [                           // Embedded Array
    { "medicine": "Atorvastatin", "dosage": "20mg", "frequency": "Daily", "days": 30 }
  ],
  "billing": { "room_charges": 1600.0, "consultation_fee": 450.0, "total_amount": 2330.0 },
  "clinical_embedding": [0.124, -0.453, 0.891, ... 64 floats] // Vector Search
}
```

3. Schema Validation Rules ($jsonSchema) & Indexing

Data safety is enforced directly at the MongoDB engine layer via `$jsonSchema`. Any insert or update violating the specification is rejected with a `WriteError`:

Field Name	Constraint Type	Rule Specification	Clinical Purpose
`patient_id`	String (Regex)	Pattern: `^PAT-[0-9]{4}$`	Standardized hospital identifier
`age`	BSON Integer	minimum: 0, maximum: 125	Prevents erroneous data entry
`gender`	Enum	['Male', 'Female', 'Other']	Controlled demographic taxonomy
`blood_group`	Enum	['A+', 'A-', 'B+', 'B-', ...]	Prevents blood bank mismatch
`vitals_history.heart_rate`	BSON Integer	minimum: 30, maximum: 240	Physiological sanity boundary
`vitals_history.oxygen_level`	BSON Integer	minimum: 50, maximum: 100	Valid SpO2 percentage range

Index Architecture:

- **Unique Indexes:** `{'patient_id': 1}` and `{'doctor_id': 1}` provide $O(1)$ lookup complexity and prevent duplicate records.
- **Compound Index:** `{'department': 1, 'status': 1}` accelerates multi-parameter directory lookups by admitted unit.
- **Multikey Index:** `{'vitals_history.heart_rate': 1}` indexes embedded array scalars for fast emergency triage filtering.

4. CRUD Operations & Array Manipulation

4.1 CREATE: Document Insertion with Embedding Synthesis

Inserts a new patient record into `patients`. The system validates schema compliance and synthesizes a 64-dimensional semantic embedding for AI search:

```
result = db.patients.insert_one(patient_document)
# Handled: DuplicateKeyError, WriteError (DocumentValidationFailure)
```

4.2 READ: Simultaneous Array Condition Matching via $elemMatch

Direct array queries match if condition A holds on one element and condition B on another. `$elemMatch` enforces that **both conditions are met simultaneously on the exact same vital reading**:

```
critical_cases = db.patients.find({
  "vitals_history": {
    "$elemMatch": {
      "heart_rate": {"$gte": 100},       // Tachycardia
      "oxygen_level": {"$lte": 95}       // Hypoxemia (Simultaneous!)
    }
  }
})
```

4.3 UPDATE: Atomic Operators ($push and $inc)

```
// 1. Append vital reading to embedded array without rewriting parent document:
db.patients.update_one({"patient_id": id}, {"$push": {"vitals_history": new_vital_check}})

// 2. Atomic numeric increment on billing ledger:
db.patients.update_one({"patient_id": id}, {"$inc": {"billing.medicine_charges": 50.0, "billin
g.total_amount": 50.0}})
```

4.4 DELETE: Soft Deletion (HIPAA & Clinical Audit Compliance)

Physical deletion destroys legal medical history. MediTrack utilizes logical soft deletion via `status: 'Archived'` and `is_archived: True`, excluding records from operational queries.

5. Advanced Aggregation Pipelines

The MongoDB Aggregation Framework processes documents through an ordered multi-stage pipeline, executing native C++ transformations inside the database engine. Five pipelines were built:

Pipeline	Key Operators	Operational & Clinical Intelligence Delivered
P1: Department Revenue & Demographics	`$match, $group, $sort`	Aggregates patient census, average age, total billing collections, and average bill amount grouped by department.
P2: Doctor Workload Attribution	`$lookup, $project, $sort`	Performs left outer join across `doctors` and `patients` to resolve caseload and revenue contribution.
P3: Clinical Critical Vitals Triage	`$unwind, $match, $sort`	Flattens embedded vitals to alert on individual historical cardiac and respiratory threshold violations (HR ≥ 100, SpO2 ≤ 95).
P4: Pharmacy Prescription Demand	`$unwind, $group, $sort`	Analyzes drug dispensing frequency and average treatment durations to optimize hospital pharmacy inventory.
P5: Executive KPI Dashboard	`$facet`	Executes bed occupancy distribution and payment realization pipelines in a single database round trip.

Pipeline 2 Highlight: Cross-Collection JOIN ($lookup)

```
db.doctors.aggregate([
  { $lookup: { from: "patients", localField: "doctor_id", foreignField: "assigned_doctor_id",
as: "cases" } },
  { $project: { name: 1, department: 1, patient_count: { $size: "$cases" }, total_rev: { $sum:
 "$cases.billing.total_amount" } } },
  { $sort: { patient_count: -1 } }
])
```

6. MongoDB as a Vector Database & AI Clinical Assistant

A standout feature of MediTrack is utilizing MongoDB as a unified **Operational and Vector Database**. Traditional AI architectures require synchronizing MongoDB with external vector stores like Pinecone or Milvus via complex ETL pipelines, leading to synchronization drift. In MediTrack, transactional health records, streaming vitals, and vector embeddings reside in the exact same document.

Vector Search Architecture:

- **Vector Representation:** Clinical symptoms and medical history are projected into a 64-dimensional dense semantic vector space normalized to unit length.

- **BSON Storage:** Stored natively in the document as an array of IEEE 754 doubles: `clinical_embedding: [0.124, -0.453, ...]`.
- **Cosine Similarity Search:** Computes similarity score $S(Q, D) = (Q \cdot D) / (\|Q\| \|D\|)$ against all inpatient charts.
- **Production Atlas Syntax:** In MongoDB Atlas, this maps natively to the `$vectorSearch` stage using HNSW graphs:

```
db.patients.aggregate([
  { $vectorSearch: {
      index: "clinical_vector_index", path: "clinical_embedding",
      queryVector: [0.124, -0.453, ...], numCandidates: 50, limit: 5
  }},
  { $project: { name: 1, department: 1, medical_notes: 1, score: { $meta: "vectorSearchScore"
} } }
])
```

7. Web Application Architecture & Verified Outputs

MediTrack provides a responsive web interface built on Flask, Tailwind CSS, and Chart.js. The table below documents verified outputs obtained during automated test execution:

Pipeline / Operation	Key Output Obtained	Status
Schema Validation Enforcement	Blocked invalid ID `'INVALID_ID_FORMAT'` and age `155` with Schema Violation Error.	**PASSED**
Pipeline 1: Department Financials	Orthopedics: $4,920 (2 patients) | Cardiology: $3,835 (3 patients) | Neurology: $1,950 (2 patients).	**PASSED**
Pipeline 2: Doctor Workload ($lookup)	Dr. Sarah Jenkins: 3 patients, $3,835 rev | Dr. Emily Taylor: 2 patients, $4,920 rev | Dr. Rajesh Sharma: 2 patients, $1,950 rev.	**PASSED**
Pipeline 3: Vitals Anomaly Alert	Detected: PAT-1005 (HR 118 BPM, SpO2 93%), PAT-1004 (HR 115 BPM, SpO2 95%), PAT-1001 (HR 105 BPM, SpO2 94%).	**PASSED**
Pipeline 4: Prescriptions ($unwind)	Top Drugs: Metoprolol (2 rx, 14.5 days avg), Aspirin (1 rx), Atorvastatin (1 rx), Betahistine (1 rx).	**PASSED**
Pipeline 5: Executive Facets ($facet)	Bed Status: 5 Admitted, 2 Observation, 1 Discharged | Payment: $6,295 Paid, $3,940 Insurance, $2,095 Pending.	**PASSED**
AI Vector Similarity Search	Query: *'chest pain and elevated pulse'* → Matched PAT-1001 (88.4%) & PAT-1005 (82.1%) with Cardiology insight.	**PASSED**

8. Conclusion & Academic Learning Outcomes

The MediTrack project successfully demonstrates the design and deployment of an enterprise-grade MongoDB application tailored to real-world healthcare informatics. By synthesizing a hybrid document structure, database-level validation, complex aggregation pipelines, and embedded vector search, the project satisfies 100% of the course outcomes (CO4, CO5, CO6) and establishes MongoDB as an agile, AI-ready operational data platform.

Student Signature:

_____

Date: October 18, 2026

Faculty Evaluator Signature:

_____

Marks Awarded: _____ / 30